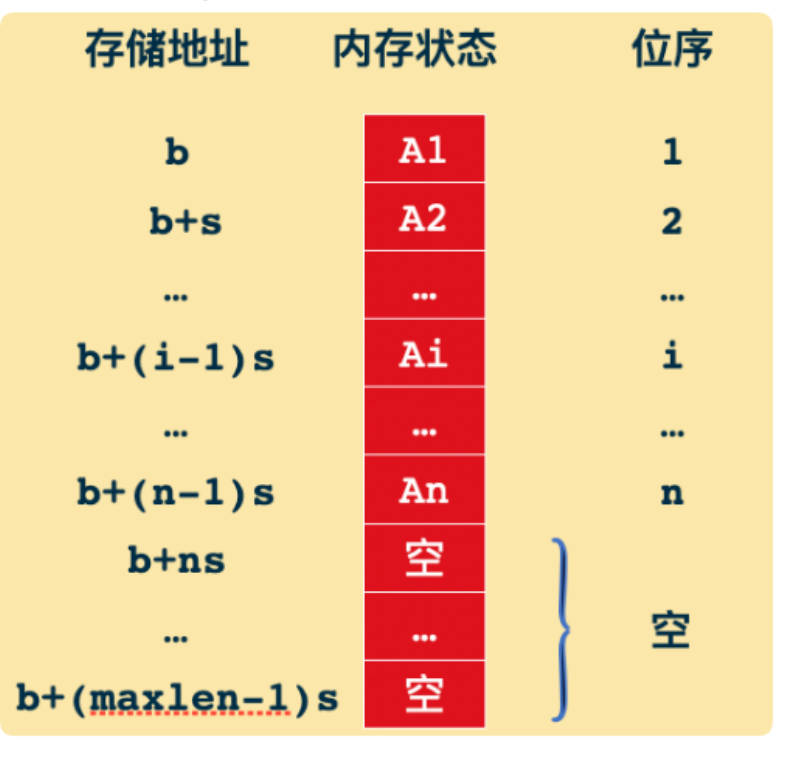
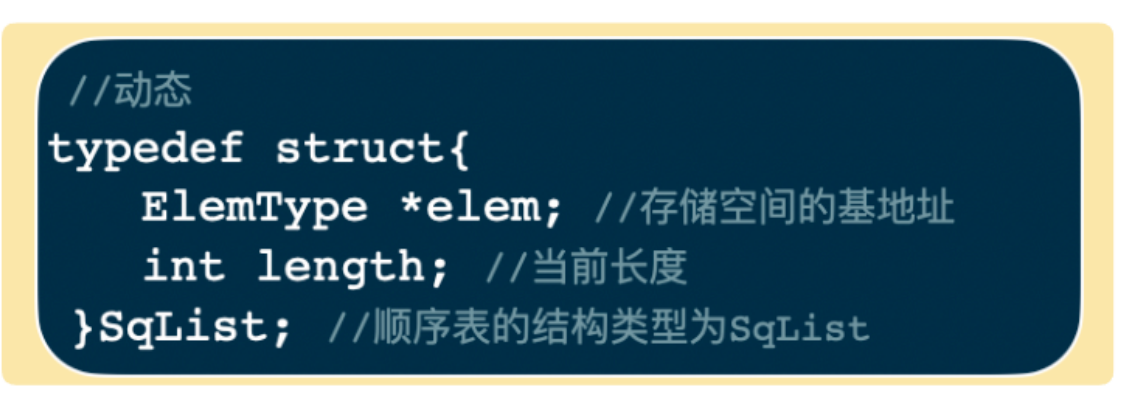
线性表

- **线性表**
  - 定义 —— 线性表是由n（n>=0）个数据特性相同的元素构成的有限序列
  - 特性
    - 线性表是一种逻辑结构
    - 表中元素的个数有限
    - 表中数据元素类型相同
    - 除第一个元素外，每个数据元素均只有一个前驱
    - 除最后一个元素外，每个数据元素均只有一个后继
  - 基本操作
    - InitList（&L） —— 初始化表，构造一个空线性表
    - Length（L） —— 求表长，返回线性表L的长度，即L中数据元素的个数
    - LocateElem（L,e） —— 按值查找操作，在表L中查找具有给定关键字值的元素
    - GetElem（L,i） —— 按位查找操作，获取表L中第i个位置的元素的值
    - ListInsert（&L,i,e） —— 插入操作，在表L中的第i个位置上插入指定元素e
    - PrintList（L） —— 输出操作，按前后顺序输出线性表L的所有元素值
    - ListDelete（&L,i,&e） —— 删除操作，删除表L中第i个位置的元素，并用e返回删除元素的值
    - Empty（L） —— 判空操作，若L为空表，则返回true，否则返回false
    - DestroyList（&L） —— 销毁操作，销毁线性表，并释放线性表L所占用的内存空间
- **线性表的顺序存储**
  - 顺序表的定义
    - **顺序表中，逻辑上相邻的数据元素物理位置也是相邻的**

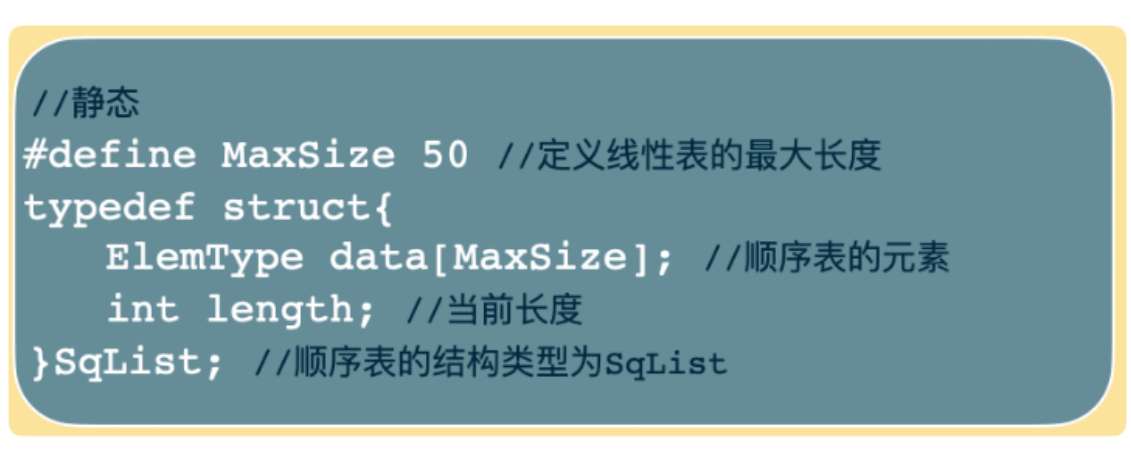


存储地址	内存状态	位序
b	A1	1
b+s	A2	2
…	…	…
b+(i-1)s	Ai	i
…	…	…
b+(n-1)s	An	n
b+ns	空	空
…	…	
b+(maxlen-1)s	空	

    - 计算顺序表中Ai+1的地址
      - Array[i] = Array[0] + i * L
      - 顺序表一般使用数组存储，所以A1对应Array[0]
    - 顺序表拥有随机存取的特性
  - 结构体

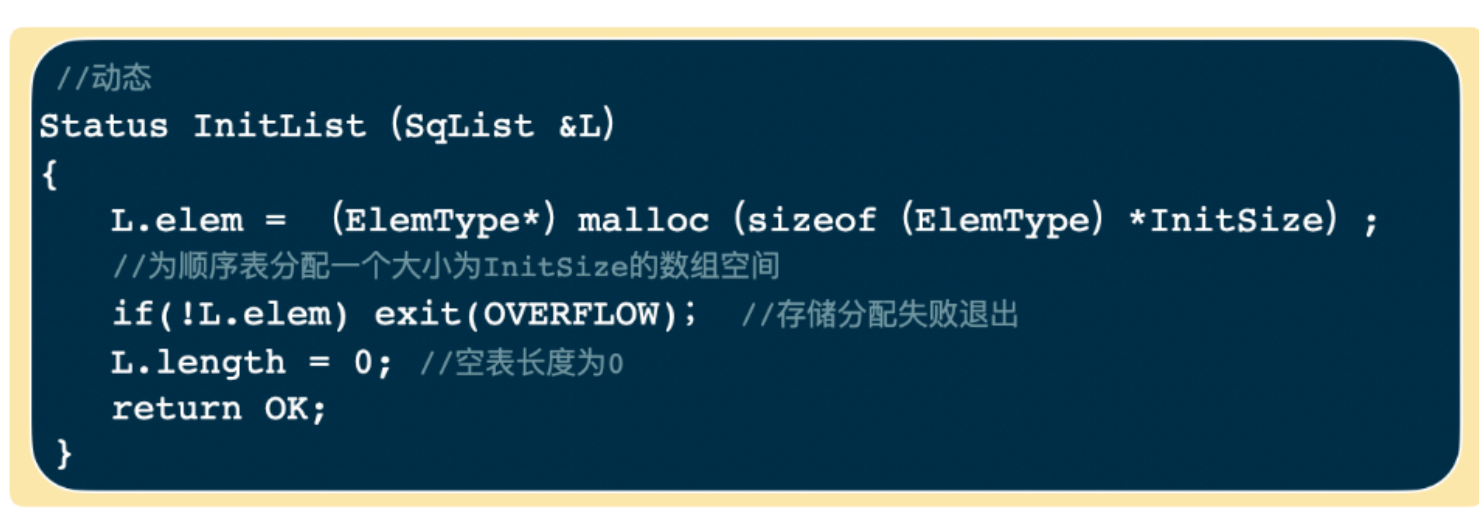


```
//动态
typedef struct{
    ElemType *elem; //存储空间的基地址
    int length; //当前长度
}SqList; //顺序表的结构类型为SqList
```



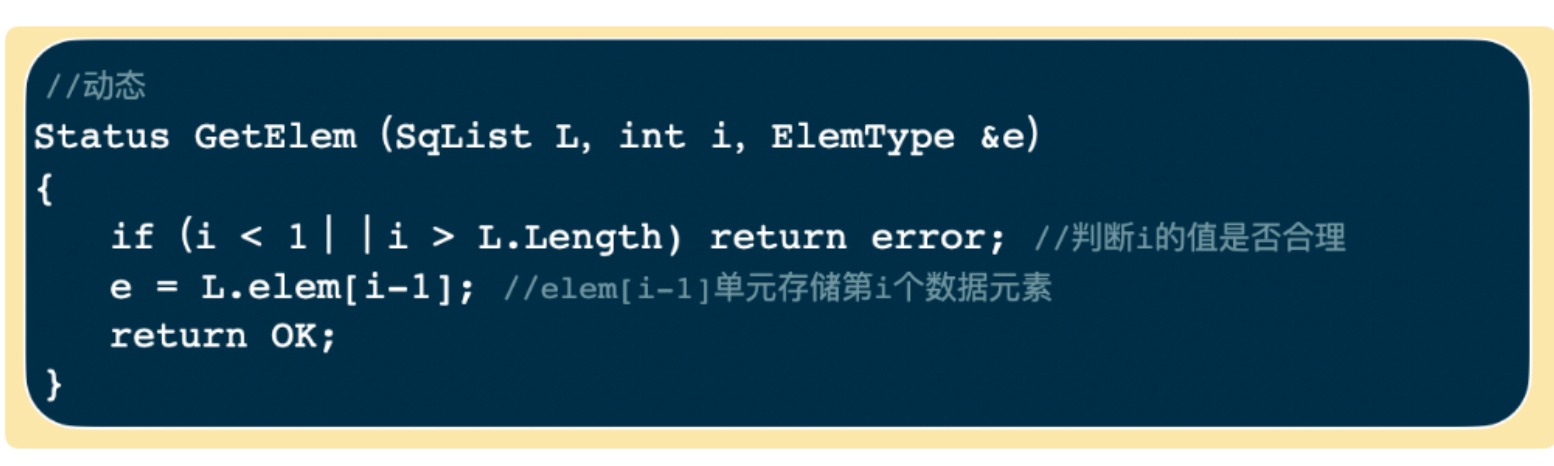
```
//静态
#define MaxSize 50 //定义线性表的最大长度
typedef struct{
   ElemType data[MaxSize]; //顺序表的元素
   int length; //当前长度
}SqList; //顺序表的结构类型为SqList
```

  - 基本操作
    - 初始化



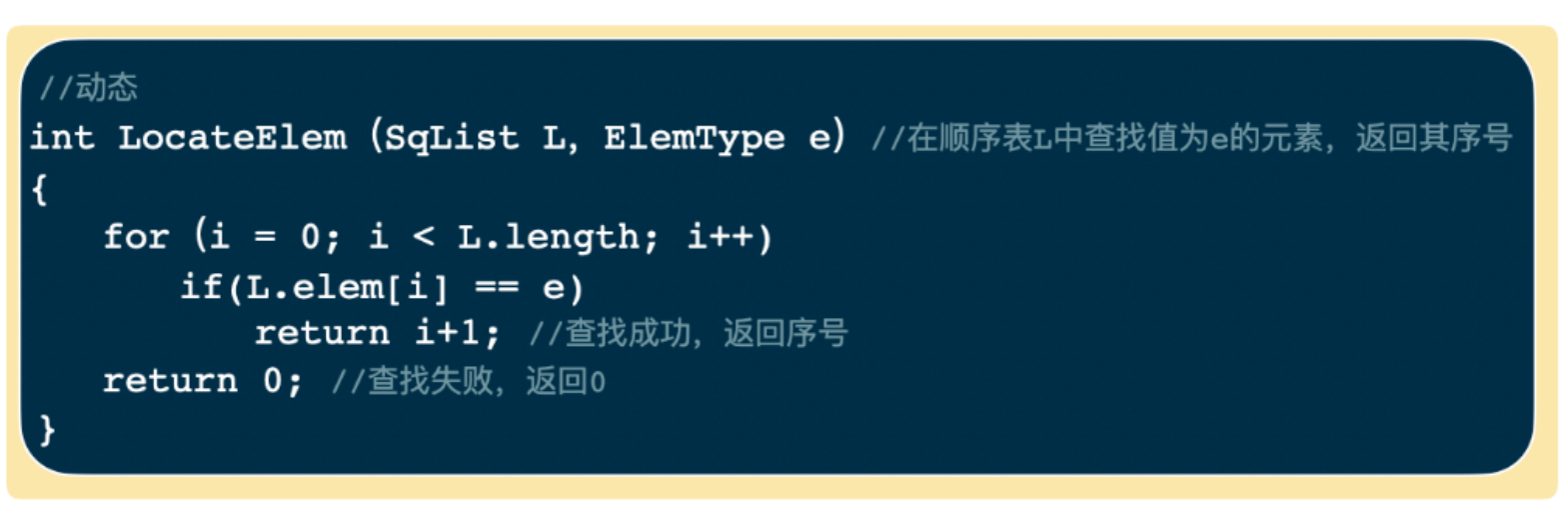
```
//动态
Status InitList (SqList &L)
{
    L.elem =  (ElemType*) malloc (sizeof (ElemType) *InitSize) ;
    //为顺序表分配一个大小为InitSize的数组空间
    if(!L.elem) exit(OVERFLOW);  //存储分配失败退出
    L.length = 0; //空表长度为0
    return OK;
}
```

    - 取值



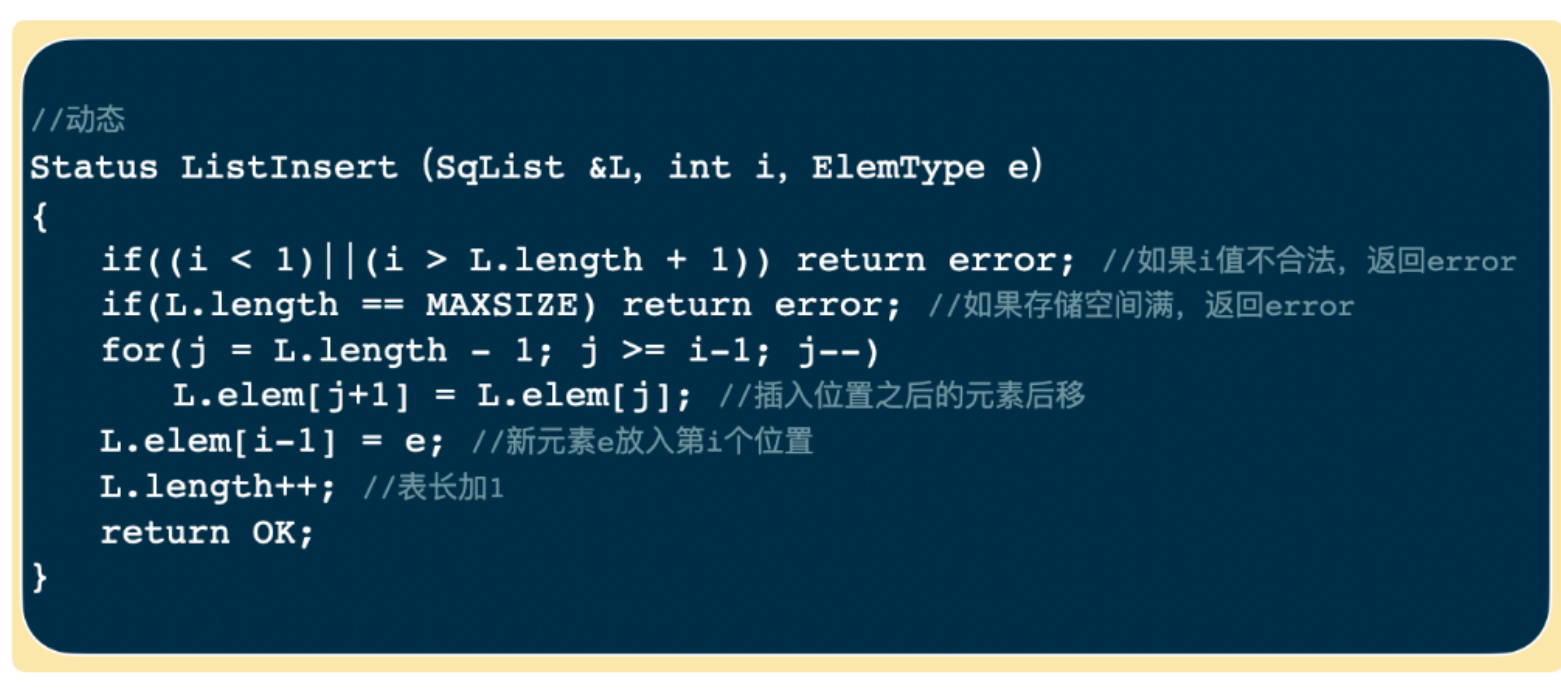
```
//动态
Status GetElem (SqList L, int i, ElemType &e)
{
   if (i < 1 | | i > L.Length) return error; //判断i的值是否合理
   e = L.elem[i-1]; //elem[i-1]单元存储第i个数据元素
   return OK;
}
```

    - 按值查找



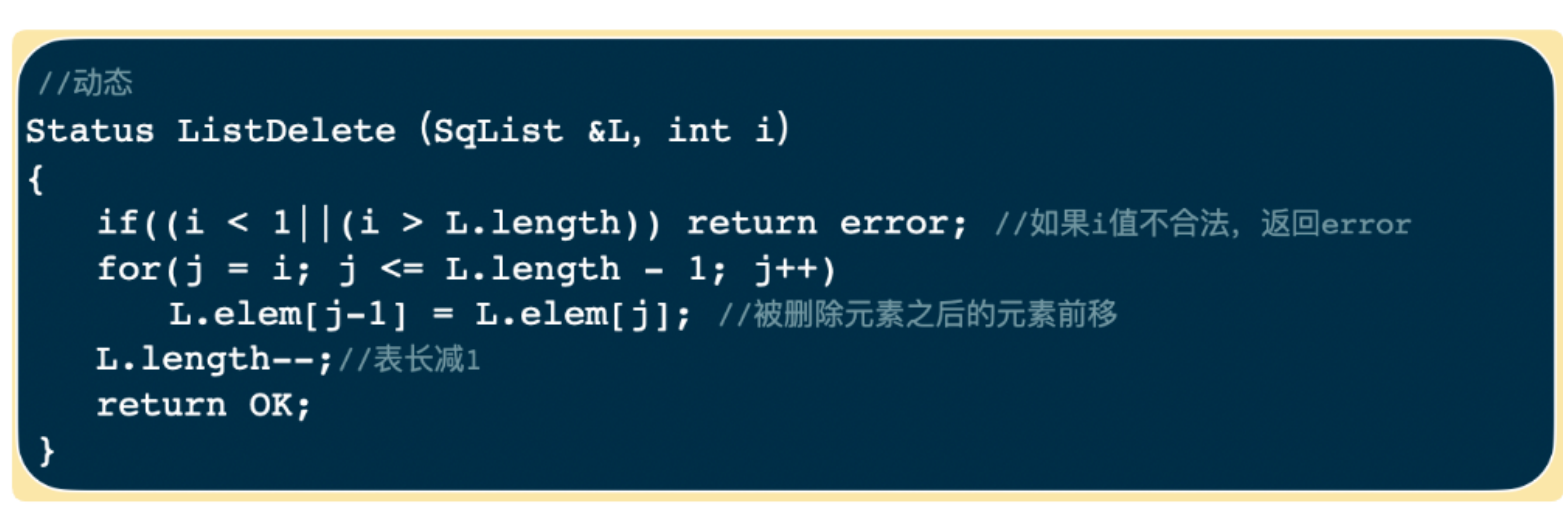
```
//动态
int LocateElem (SqList L, ElemType e) //在顺序表L中查找值为e的元素，返回其序号
{
   for (i = 0; i < L.length; i++)
       if(L.elem[i] == e)
           return i+1; //查找成功，返回序号
   return 0; //查找失败，返回0
}
```

    - 插入



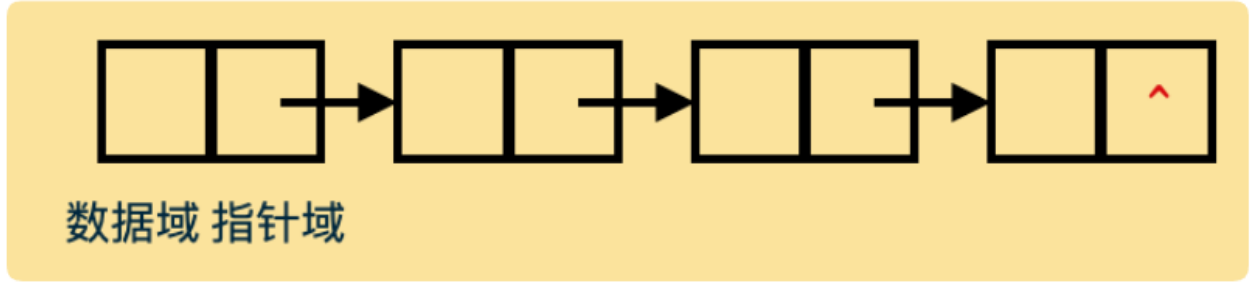
```
//动态
Status ListInsert (SqList &L, int i, ElemType e)
{
   if((i < 1)||(i > L.length + 1)) return error; //如果i值不合法，返回error
   if(L.length == MAXSIZE) return error; //如果存储空间满，返回error
   for(j = L.length - 1; j >= i-1; j--)
       L.elem[j+1] = L.elem[j]; //插入位置之后的元素后移
   L.elem[i-1] = e; //新元素e放入第i个位置
   L.length++; //表长加1
   return OK;
}
```

    - 删除

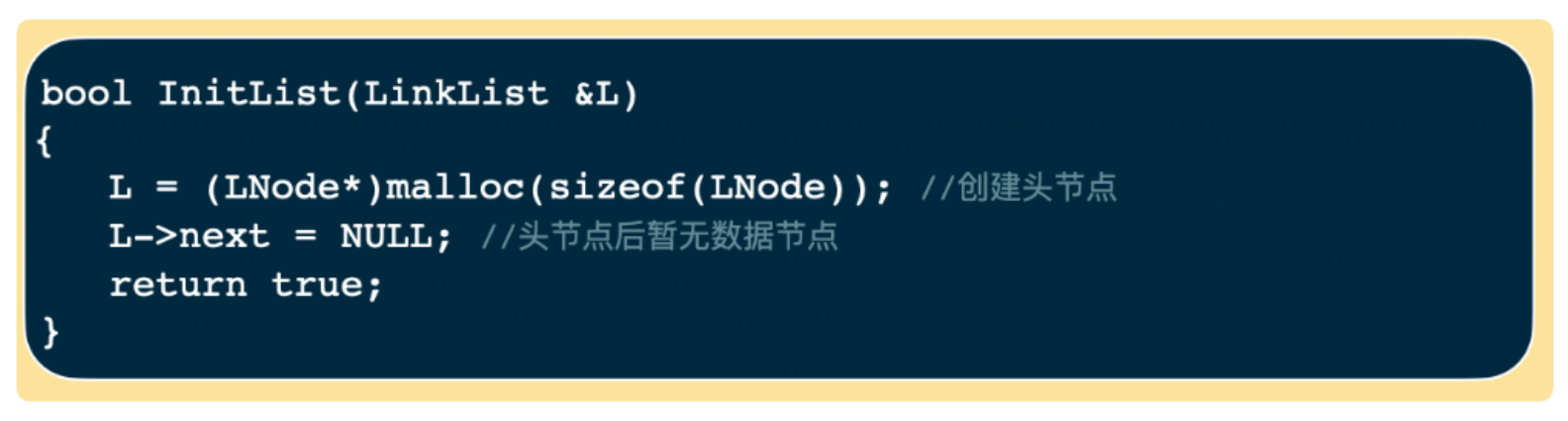


```
//动态
Status ListDelete (SqList &L, int i)
{
   if((i < 1||(i > L.length)) return error; //如果i值不合法，返回error
   for(j = i; j <= L.length - 1; j++)
       L.elem[j-1] = L.elem[j]; //被删除元素之后的元素前移
   L.length--;//表长减1
   return OK;
}
```

- **线性表的链式存储**
  - 单链表的定义

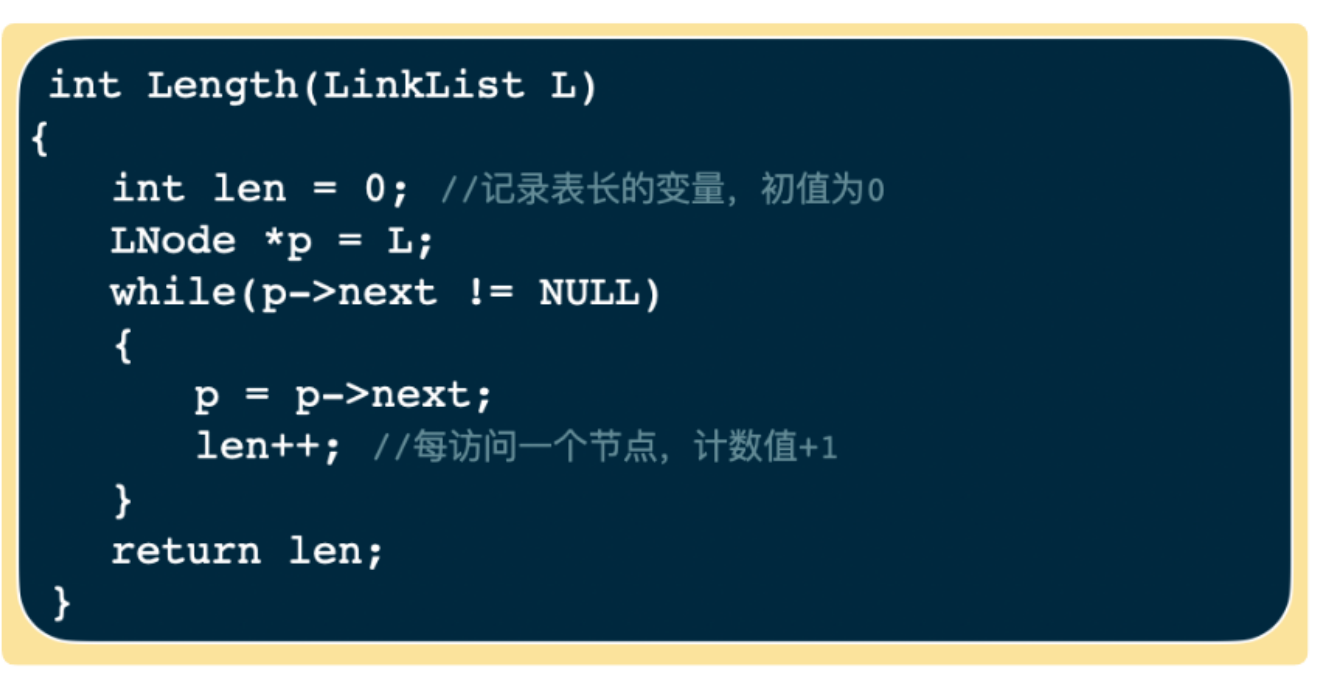


    - 数据域 —— 存放数据元素的信息
    - 指针域 —— 存放后继节点的地址
    - 单链表不再依赖连续内存空间，但每个节点需要存储一个额外的指针，增加了存储开销
    - 单链表不支持随机存取
  - 头节点的作用
    - 便于首元节点的处理
    - 便于空表和非空表的统一处理
  - 基本操作
    - 初始化



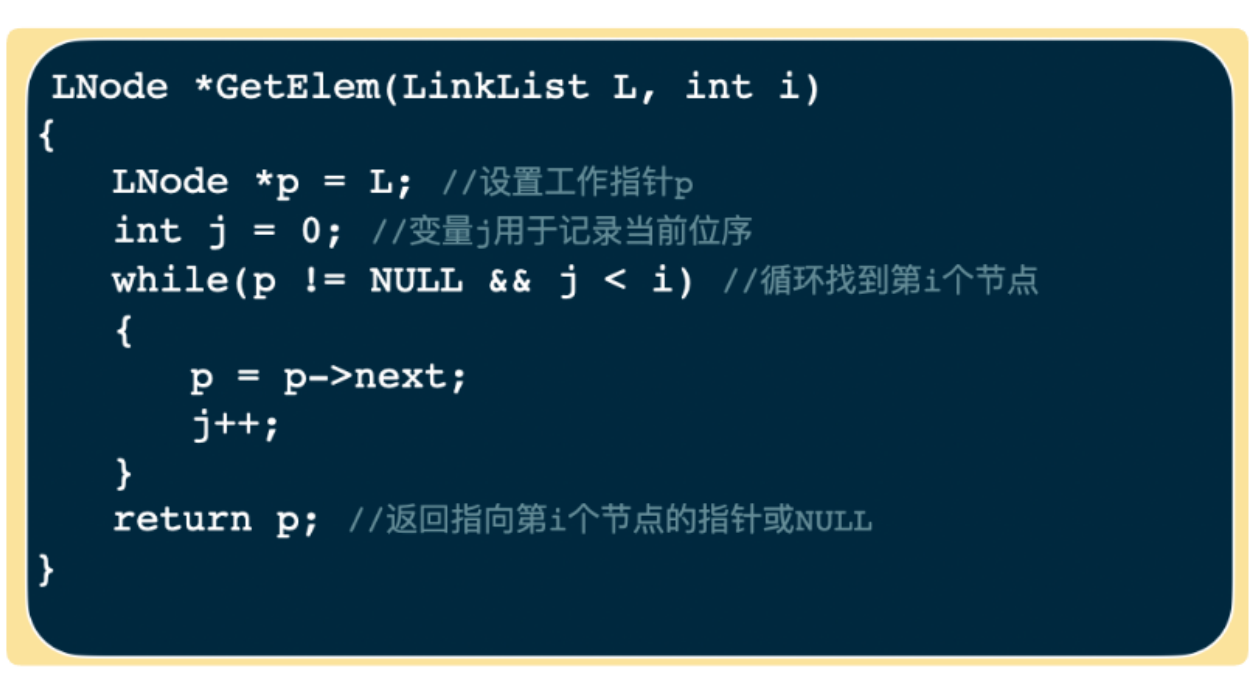
```
bool InitList(LinkList &L)
{
   L = (LNode*)malloc(sizeof(LNode)); //创建头节点
   L->next = NULL; //头节点后暂无数据节点
   return true;
}
```

    - 求表长



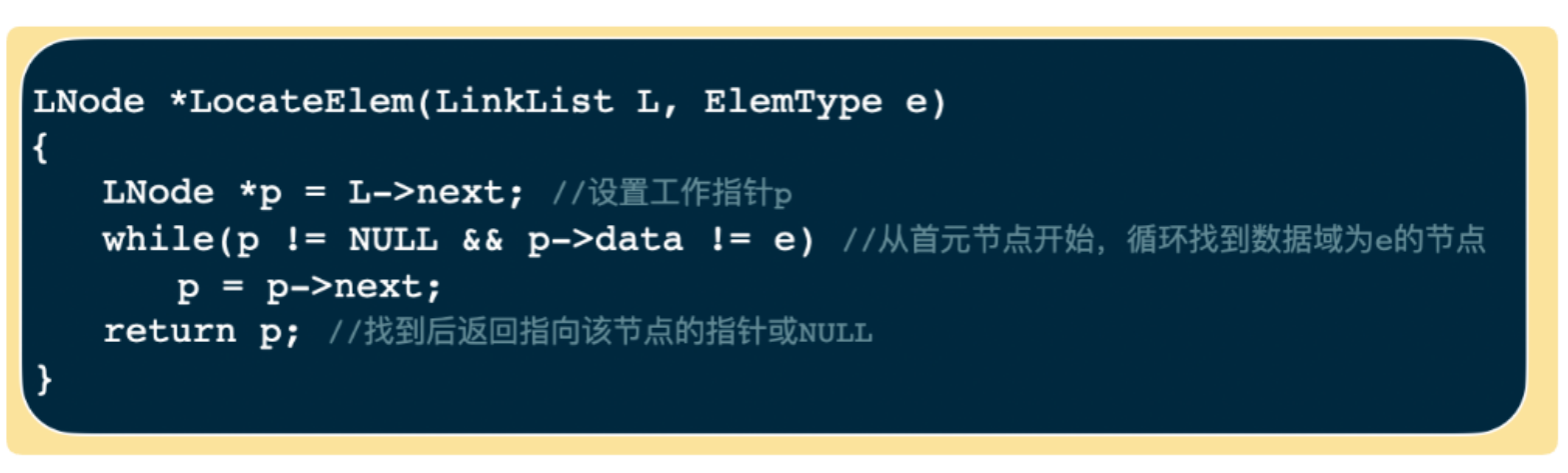
```
int Length(LinkList L)
{
   int len = 0; //记录表长的变量，初值为0
   LNode *p = L;
   while(p->next != NULL)
   {
      p = p->next;
      len++; //每访问一个节点，计数值+1
   }
   return len;
}
```

    - 按序查找



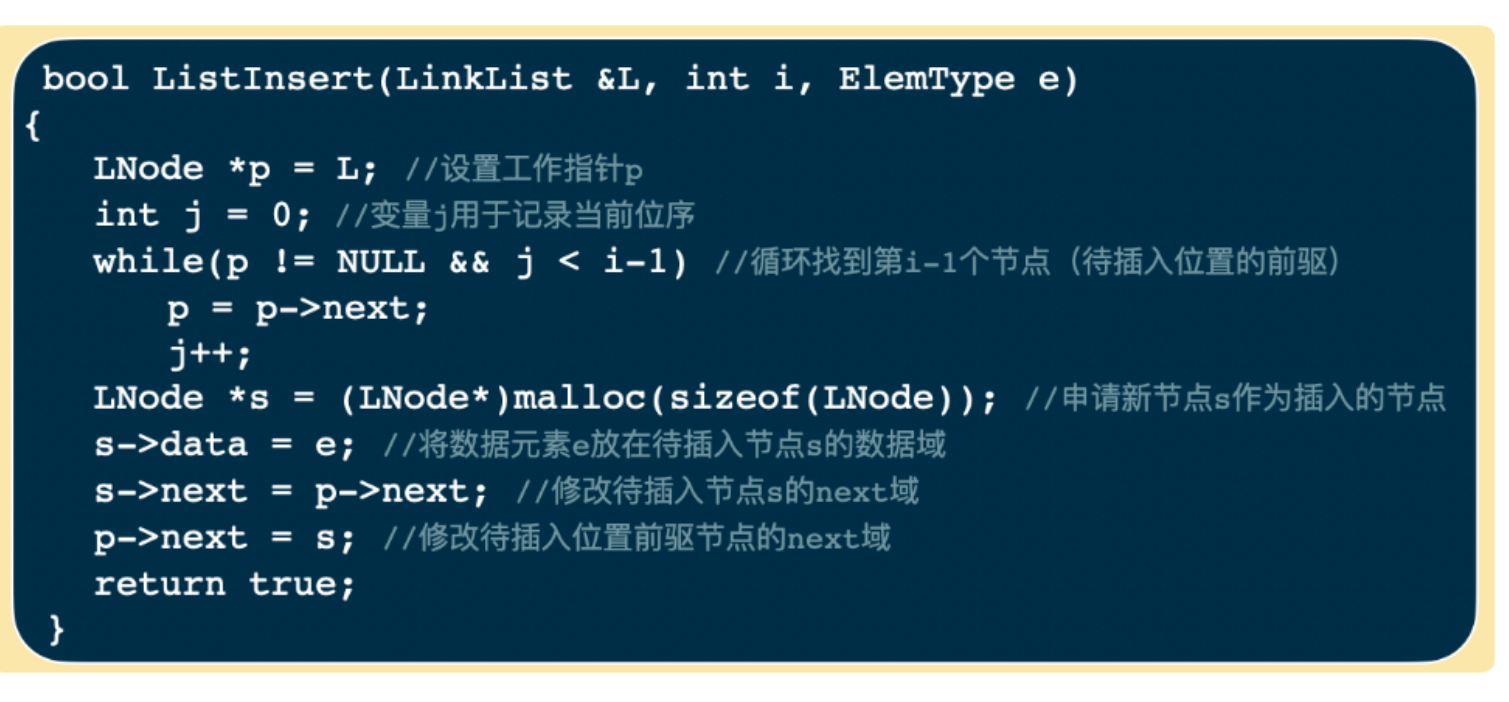
```
LNode *GetElem(LinkList L, int i)
{
   LNode *p = L; //设置工作指针p
   int j = 0; //变量j用于记录当前位序
   while(p != NULL && j < i) //循环找到第i个节点
   {
      p = p->next;
      j++;
   }
   return p; //返回指向第i个节点的指针或NULL
}
```

    - 按值查找



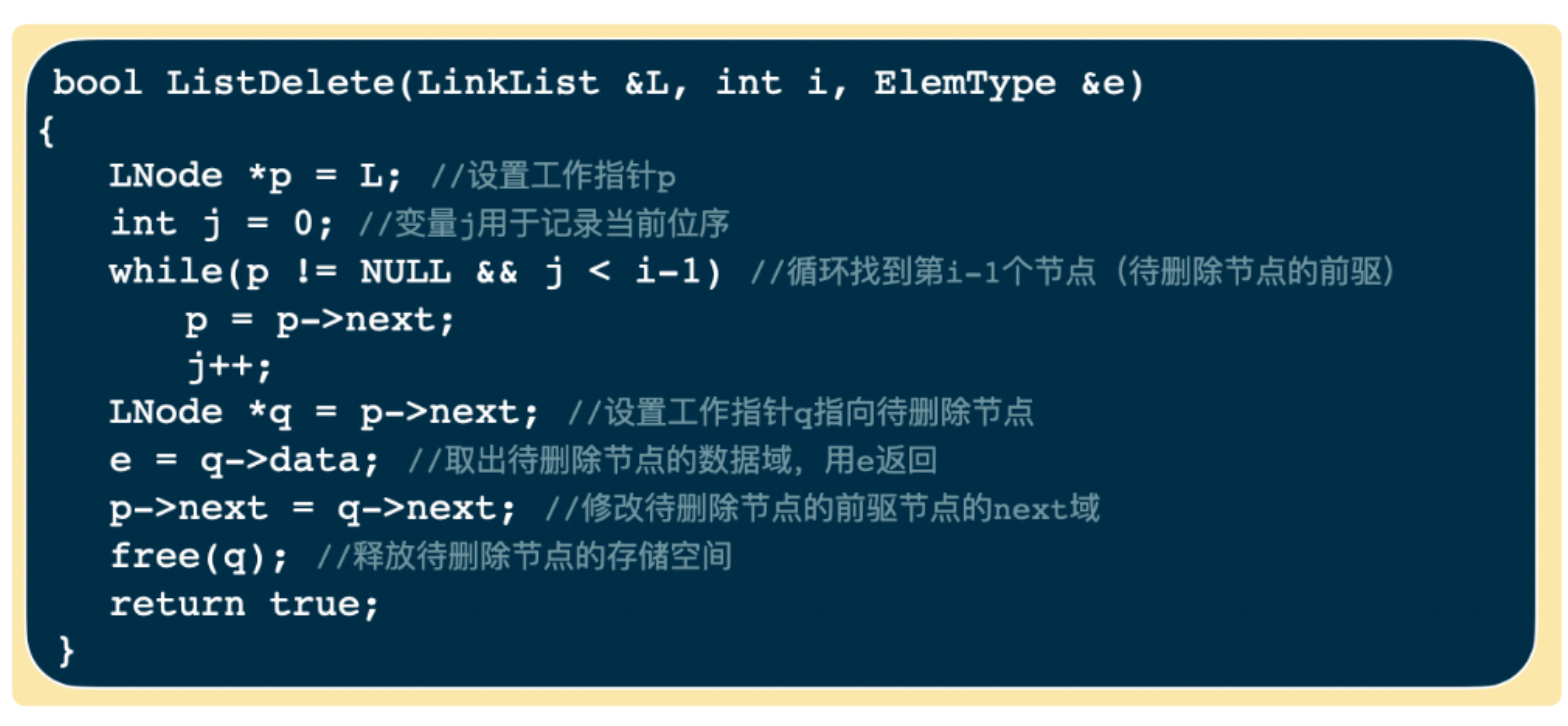
```
LNode *LocateElem(LinkList L, ElemType e)
{
   LNode *p = L->next; //设置工作指针p
   while(p != NULL && p->data != e) //从首元节点开始，循环找到数据域为e的节点
      p = p->next;
   return p; //找到后返回指向该节点的指针或NULL
}
```

    - 插入



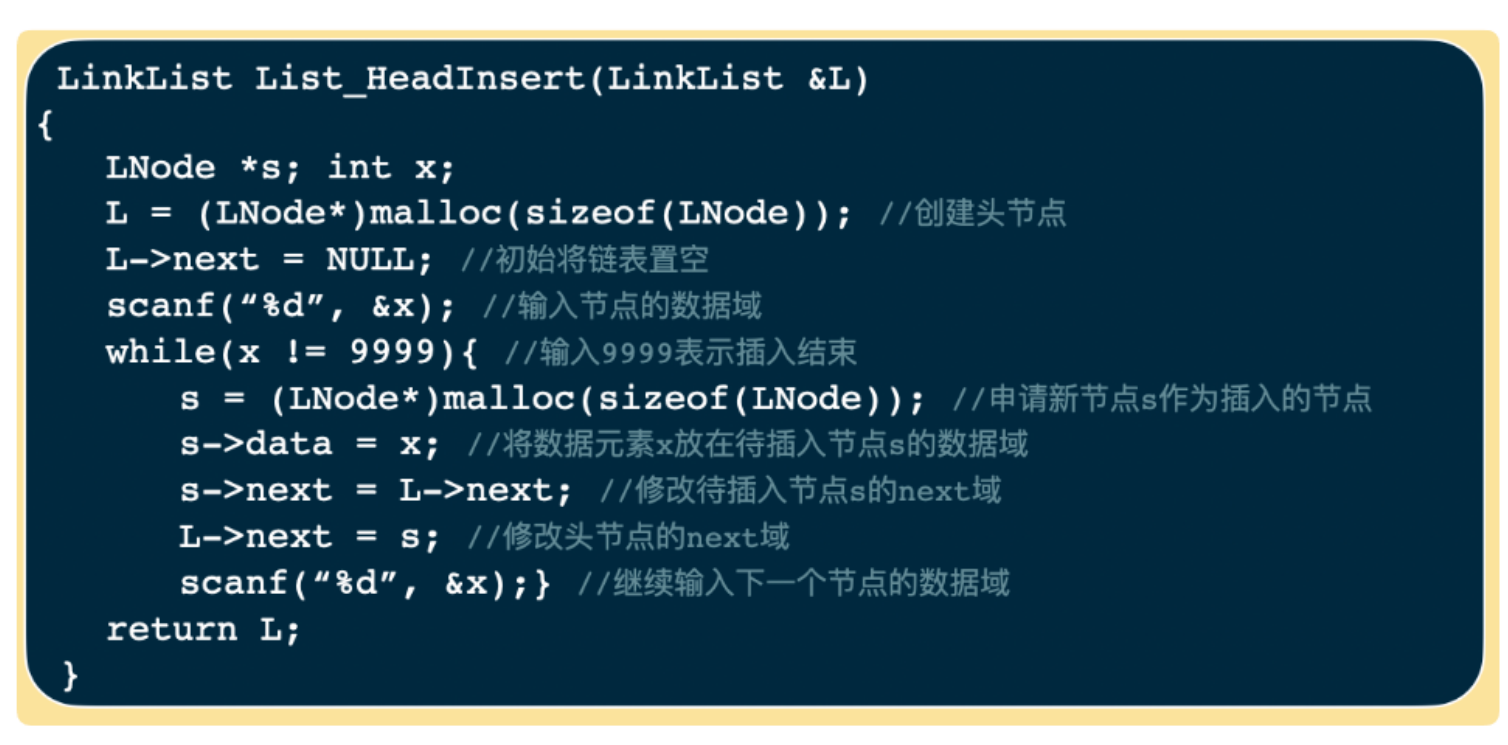
```
bool ListInsert(LinkList &L, int i, ElemType e)
{
   LNode *p = L; //设置工作指针p
   int j = 0; //变量j用于记录当前位序
   while(p != NULL && j < i-1) //循环找到第i-1个节点（待插入位置的前驱）
      p = p->next;
      j++;
   LNode *s = (LNode*)malloc(sizeof(LNode)); //申请新节点s作为插入的节点
   s->data = e; //将数据元素e放在待插入节点s的数据域
   s->next = p->next; //修改待插入节点s的next域
   p->next = s; //修改待插入位置前驱节点的next域
   return true;
}
```

    - 删除



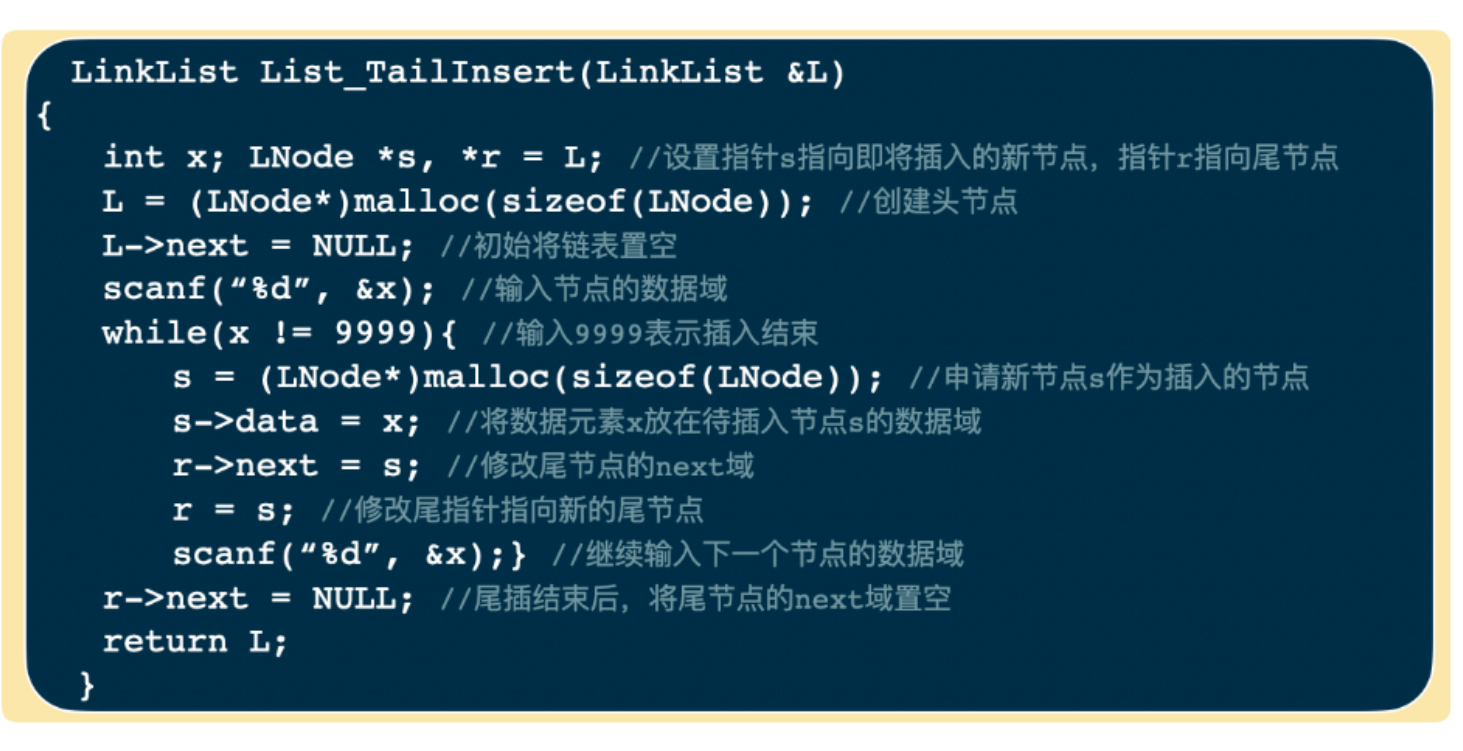
```
bool ListDelete(LinkList &L, int i, ElemType &e)
{
   LNode *p = L; //设置工作指针p
   int j = 0; //变量j用于记录当前位序
   while(p != NULL && j < i-1) //循环找到第i-1个节点（待删除节点的前驱）
      p = p->next;
      j++;
   LNode *q = p->next; //设置工作指针q指向待删除节点
   e = q->data; //取出待删除节点的数据域，用e返回
   p->next = q->next; //修改待删除节点的前驱节点的next域
   free(q); //释放待删除节点的存储空间
   return true;
}
```

    - 头插法



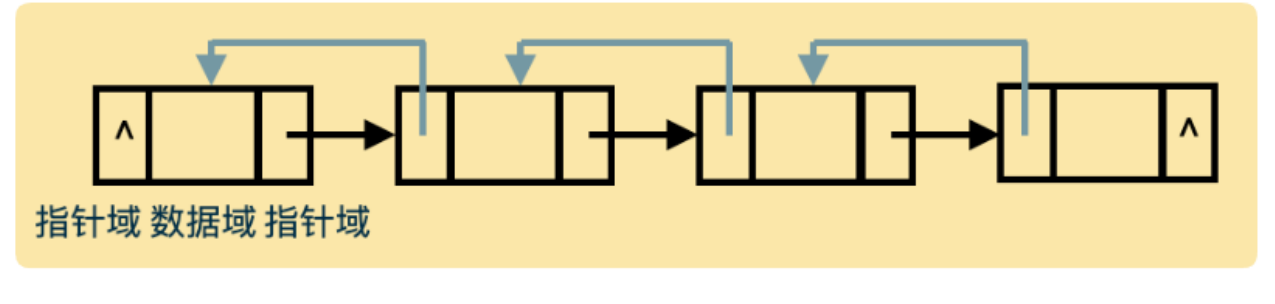
```
LinkList List_HeadInsert(LinkList &L)
{
   LNode *s; int x;
   L = (LNode*)malloc(sizeof(LNode)); //创建头节点
   L->next = NULL; //初始将链表置空
   scanf("%d", &x); //输入节点的数据域
   while(x != 9999){ //输入9999表示插入结束
      s = (LNode*)malloc(sizeof(LNode)); //申请新节点s作为插入的节点
      s->data = x; //将数据元素x放在待插入节点s的数据域
      s->next = L->next; //修改待插入节点s的next域
      L->next = s; //修改头节点的next域
      scanf("%d", &x);} //继续输入下一个节点的数据域
   return L;
}
```

    - 尾插法

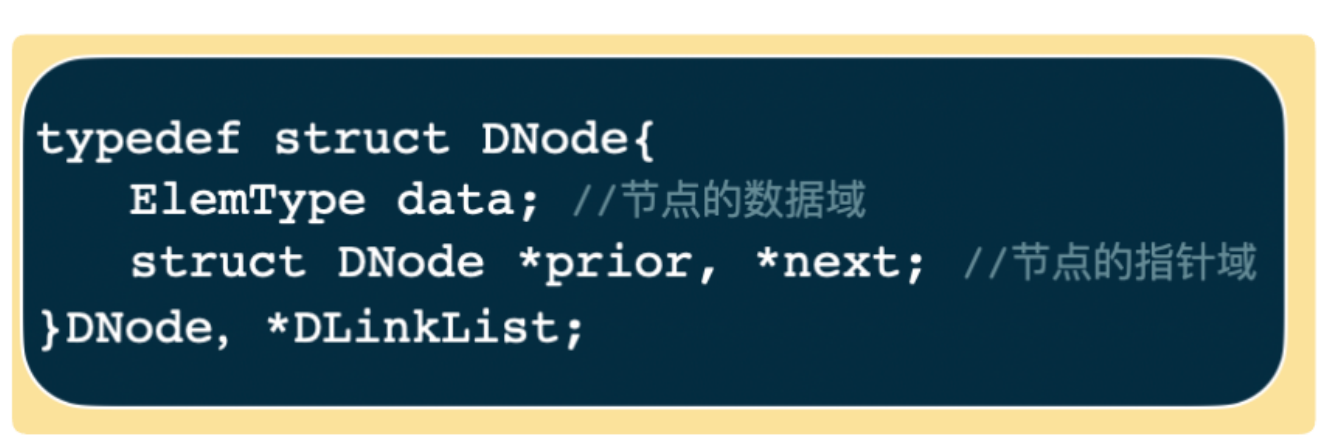


```
LinkList List_TailInsert(LinkList &L)
{
   int x; LNode *s, *r = L; //设置指针s指向即将插入的新节点，指针r指向尾节点
   L = (LNode*)malloc(sizeof(LNode)); //创建头节点
   L->next = NULL; //初始将链表置空
   scanf("%d", &x); //输入节点的数据域
   while(x != 9999){ //输入9999表示插入结束
      s = (LNode*)malloc(sizeof(LNode)); //申请新节点s作为插入的节点
      s->data = x; //将数据元素x放在待插入节点s的数据域
      r->next = s; //修改尾节点的next域
      r = s; //修改尾指针指向新的尾节点
      scanf("%d", &x);} //继续输入下一个节点的数据域
   r->next = NULL; //尾插结束后，将尾节点的next域置空
   return L;
}
```

  - 双链表
    - 定义

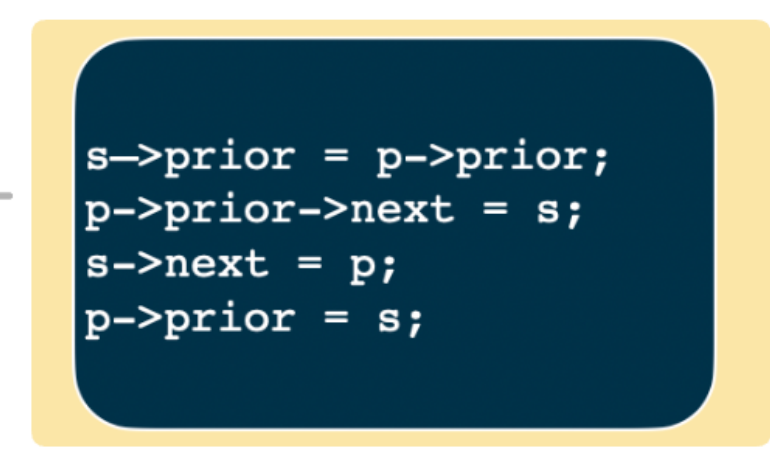


      - 数据域 —— 存放数据元素的信息
      - prior指针域 —— 存放前驱节点的地址
      - next指针域 —— 存放后继节点的地址
      - 双链表通过为每个节点增加前驱指针，使得链表可以双向遍历，能在已知结点位置时以O(1)时间完成删除与插入，弥补单链表必须遍历找前驱结点的不足
    - 结构体



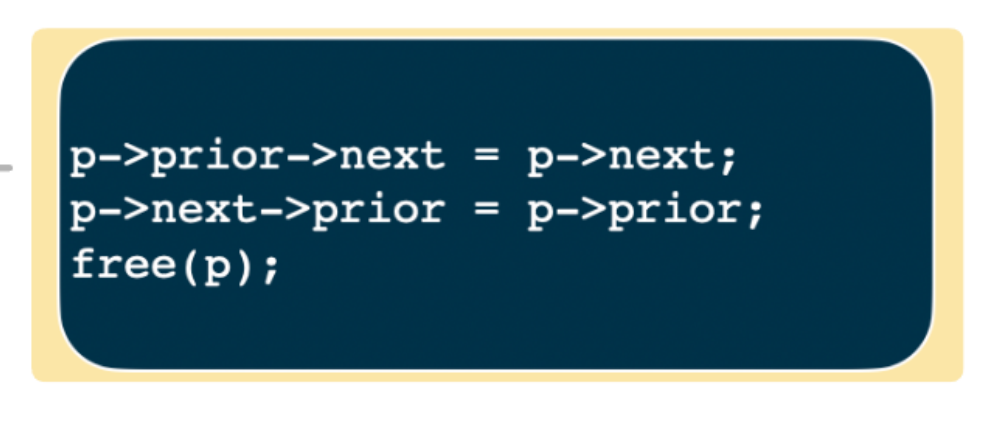
```
typedef struct DNode{
    ElemType data; //节点的数据域
    struct DNode *prior, *next; //节点的指针域
}DNode, *DLinkList;
```

    - 基本操作
      - 插入



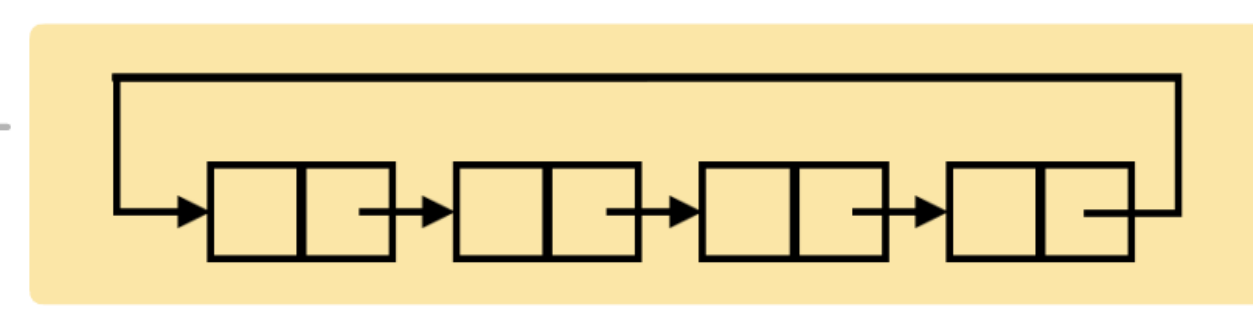
```
s->prior = p->prior;
p->prior->next = s;
s->next = p;
p->prior = s;
```

      - 删除

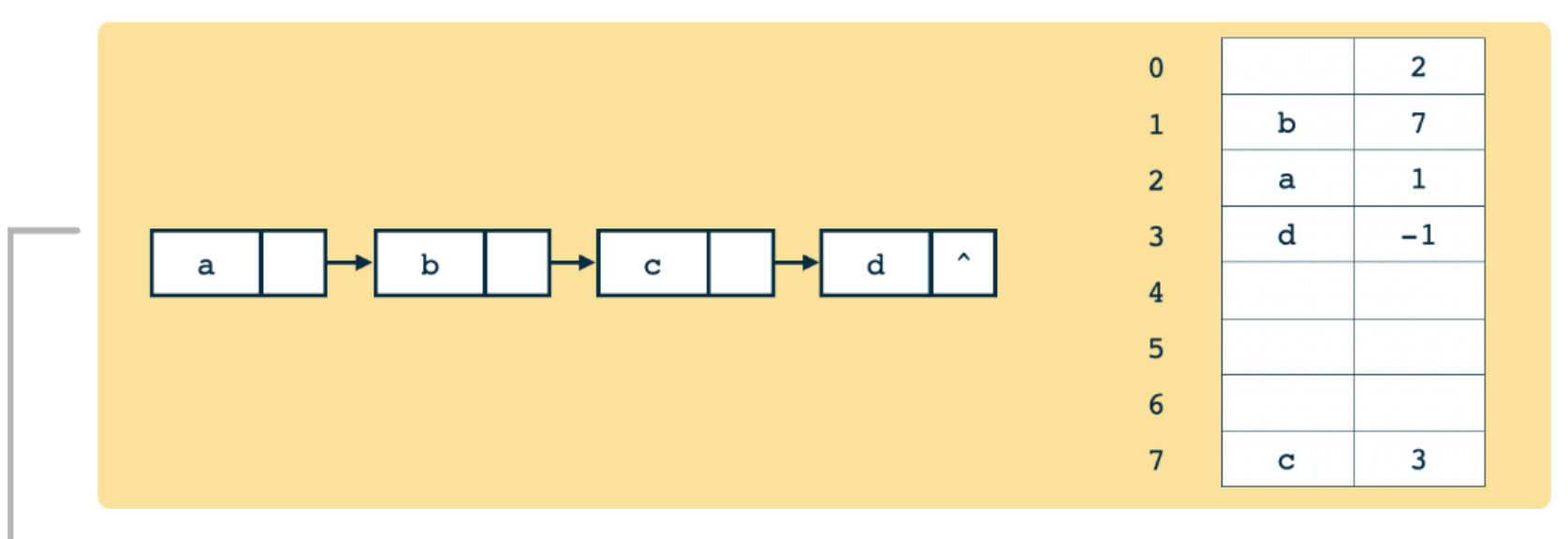


```
p->prior->next = p->next;
p->next->prior = p->prior;
free(p);
```

  - 循环链表

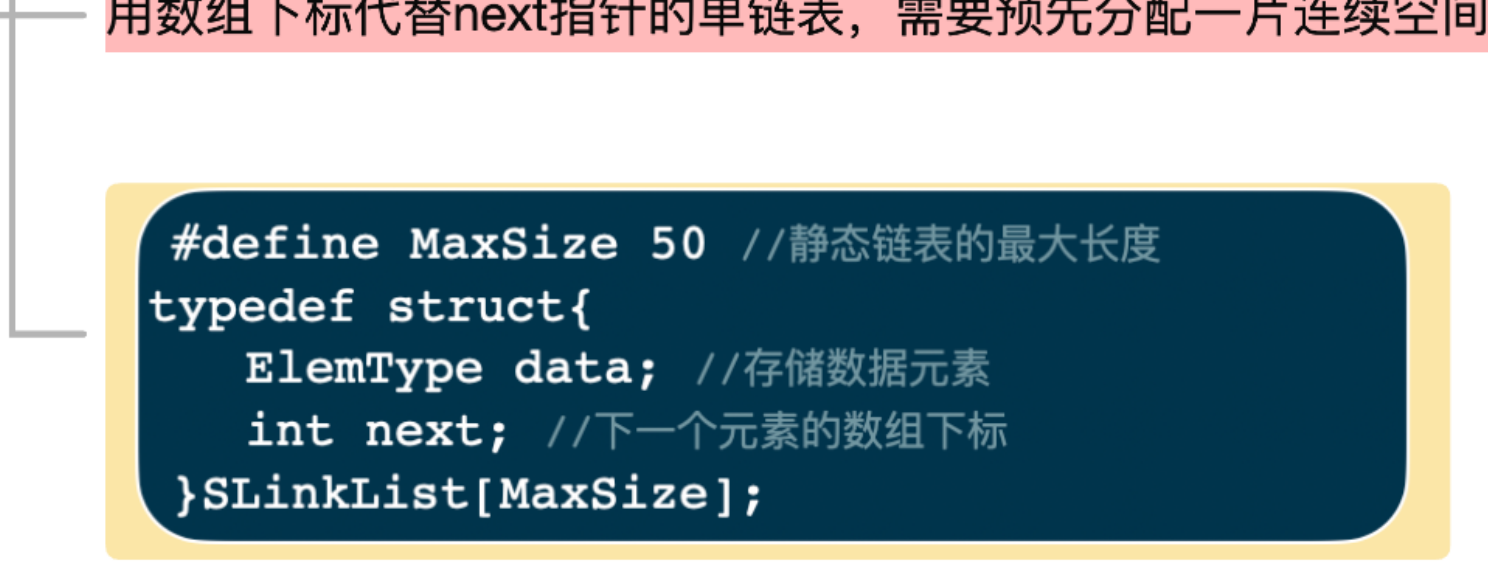


    - 循环链表把尾节点与头节点相连，使链表首尾等价，遍历无需判空且可从任意结点连续访问整个表，从而避免链表在表头表尾处的特殊处理
  - 静态链表



0		2
1	b	7
2	a	1
3	d	-1
4		
5		
6		
7	c	3

    - 用数组下标代替next指针的单链表，需要预先分配一片连续空间



```
#define MaxSize 50 //静态链表的最大长度
typedef struct{
    ElemType data; //存储数据元素
    int next; //下一个元素的数组下标
}SLinkList[MaxSize];
```

by 动物凶猛 由 幕布 发布